Arranging Shoes (Review)

Let's consider the first (leftmost) shoe A in the line and its matching shoe (that is, the shoe, that this one will be paired with in an optimal arrangement). Let's suppose that the matching shoe is initially at position k ($k \geq 1$). Each possible swap either involves moving at least one of these two shoes or does not involve moving any of them. Swaps of the first kind do not affect the relative ordering of all the other shoes. At the same time, swaps of the second kind do not affect the two shoes in question, which means that in order to pair up these two shoes there should be at least $k-1$ swaps of the first kind, in case the leftmost shoe is a left shoe, or k swaps, if it's a right shoe. Regardless of how swaps of the second kind are going to advance, we could initially perform this set of $k-1$ or k swaps, moving the shoe initially located at position k to the left until the pair occupies the two leftmost positions.

The greedy approach is optimal since the pair of shoes handled in this way will not interfere with any further swaps. Moreover, it always makes sense to take the leftmost one among the shoes matching shoe A, since otherwise the same intermediate arrangement with the two leftmost positions occupied by a matching pair of shoes could be obtained by an even shorter sequence of swaps: if another matching shoe is at position $k' > k$, then instead of moving k' all the way to the left, first move it until the shoe occupies position $k+1$, but then instead of moving it further, simply start moving the shoe that was at position k.

The process can be repeated until the arrangement of the shoes becomes valid. This can be modelled naively in quadratic time, which solves most of the subtasks, or more efficiently in $\mathcal{O}(n \log n)$ using a Fenwick tree or a segment tree.

Split (Review)

Subtask 1

In this subtask, the given graph is a path or a cycle. Therefore, the answer is always YES and a solution can be easily found by partitioning a path: $v_1, v_2, \ldots, v_n$ into $A = [v_1, v_2, \ldots, v_a]$, $B = [v_{a+1}, v_{a+2}, \ldots, v_{a+b}]$, and $C = [v_{a+b+1}, v_{a+b+2}, \ldots, v_n]$. In case of a cycle, we can cut the cycle in any place and apply the similar solution as we had in path.

Subtask 2

In this subtask, the answer is always YES as follows. Let us assume that $a \leq b \leq c$. Find a connected subgraph of size b (like using DFS or any graph traversing algorithm) as subset B, construct subset A consisting of an arbitrary vertex outside of B, and other vertices are set C.

Subtask 3

In this subtask, the given graph is a tree. The solution is similar to that of the the last subtask but the cases to be considered are easier. Without loss of generality suppose $a \leq b \leq c$. One solution is to run a DFS on an arbitrary vertex to find some arbitrary spanning tree and find a vertex v such that the size of the subtree of v [including v] denoted by $size(v)$ is at least a, but the sizes of the subtrees of all children of v are less than a. Remove the edge between v and the parent of v which partition the tree into two connected components. If both of the components have a size more than a, then the answer is YES. Otherwise, the answer is no since removing v from the original graph only produces components of size less than a. If the answer is YES, then a connected subgraph of size a (namely A) from the smallest component, and a connected subgraph of size b (namely B) from the smallest component can be extracted (since $b \leq c$, the size of the larger component is at least b). Finally, C consists of all of the remaining vertices.

Subtask 4

The input of this subtask is similar to that of the final subtask. However, since the limits for n and m are more restricted, one can use n instances of DFS instead of only one.

Subtast 5

Similar to the solution of Subtask 3, assume we run a DFS on an arbitrary vertex and found a vertex v such that $size(v) \geq a$ but the sizes of the subtrees of the children of v are all smaller than a. Note that other edges outside the DFS tree are backward edges. Hence, the children of v may have backward edges to v or the ancestors of v, but no edge exists between them. Suppose we want to partition the graph into two connected components by only considering the edges of the DFS tree and removing the edge between v and the parent of v. In this way the graph is partitioned into part P_1 consisting of v and its subtree and P_2 consisting of all other vertices. Before proceeding, we check every child of v (in an arbitrary order) like u and if the following conditions hold, we remove its subtree from P_1 and add it to P_2. The subtree of u has a backward edge to the ancestors of v. The size of P_1 after removing the subtree of u is still at least a. By doing so, one by one, we may encounter a child of v, namely u, such that $|P_1| \geq a$ and $|P_1| - size(u) < a$. Hence, $|P_2| > (n - 2a) = (a + b + c - 2a) = (b + c - a) \geq b$. In the other case, either $|P_2| > a$, which is also good since $|P_1| > n - b = (a + b + c) - b = a + c > b$, or $|P_2| < a$ and the answer is NO, since v is a cut vertex such that removing it from the original graph produces several components where all of them are of size less than a.

Rect (Review)

Subtasks 1, 2, and 3

Subtask 1 can be solved via a trivial $\mathcal{O}(n^6)$ solution: considering all $\mathcal{O}(n^4)$ rectangles and checking whether each of them is valid in $\mathcal{O}(n^2)$. Moreover, if we use breaks in loops for checking whether a rectangle is valid, then this solution is accepted in Subtask 1, 2, and 3. Moreover, an $\mathcal{O}(n^5)$ solution exist which gets accepted in Subtask 1 and 2, and an $\mathcal{O}(n^4)$ solution exist which gets accepted in Subtask 1, 2, and 3.

Subtask 5

In this subtask, valid rectangles are all consecutive subsets of the middle. All of these rectangles can be checked in time $\mathcal{O}(m^2)$.

Subtask 6

This subtask is finding a 0 rectangle with 1 on outer borders (except corners). This is a classic dynamic programming problem which can be solved in $\mathcal{O}(n^2)$. Moreover, most of the $\mathcal{O}(n^3)$ and $\mathcal{O}(n^2 \log n)$ solutions run in $\mathcal{O}(n^2)$ in this subtask.

Subtasks 4 and 7

Solution 1

We denote a pair of cells in a row or a column which are on the outer border of a valid rectangle as a good pair. The total number of good pairs are bounded by $\mathcal{O}(n^2)$ and can be found using a stack and traversing over cells of each row and each column. Then, we can extend these good pairs into two potential sides of a valid rectangle. Therefore, we have potential left and right sides, and potential top and bottom sides of valid rectangles. Finally, we try all cells as the bottom-right corner of valid rectangles and match sizes. If the matching phase is done naively, we have a $\mathcal{O}(n^3)$ solution which get accepted in subtask 4 but not Subtask 7. However, many optimizations results in getting accepted in Subtask 7. The best of which is using a Fenwick tree which decreases the total running time downto $\mathcal{O}(n^2 \log n)$. Moreover, in order to get accepted in Subtask 7, one might not use time-consuming data

structure libraries such as STL maps.

Solution 2

A very useful observation is that in a good pair, if we look at the smaller element, the pair is unique (one for the left direction, and one for the right direction). Using this observation, we can change the data structure needed for keeping good pairs into two simple arrays. For a valid rectangle, let a block corner be a 2x2 square that exactly one of its elements is inside the rectangle. Each valid rectangle have exactly four block corners. We can connect block corners of a valid rectangle which are in a row or a column based on whether which one captures the larger element in the outer border of the row or column. These connections define a structure for a valid rectangle. At most 2^4 = 16 structures exist and for each structure there is a block corner that uniquely define the valid rectangle. Therefore, the number of valid rectangles is bounded by $\mathcal{O}(n^2)$. After finding these $\mathcal{O}(n^2)$ candidate rectangles, we can check each of them in $\mathcal{O}(1)$ after a preprocess in $\mathcal{O}(n^2)$.

Line (Review)

In this problem, we are given n dots in a 2D-plane with distinct x coordinates and y coordinates. Our task is to construct a broken line starting from the origin that will cover all dots.

$2n$ solution

An easy solution is to use 2 segments to cover each dot (for example: first set the x coordinate, then y coordinate). This solution uses $2n$ segments and receives 12 points.

Spiral

Let us consider the left-most, top-most, right-most and bottom-most dot. They create a bounding box which we can cover with 4 segments. We can remove these points and continue with a smaller problem. To connect the bounding boxes, we can just continue the line from the inner bounding box and go toward the proper direction in the outer box. This solution receives about 50 points, depending on the implementation.

Chain

Let us notice that we can cover a chain of points (for example with increasing x and y values) without wasting any segment. From the Dilworth's theorem, we know that in the sequence of length k, there exists an increasing sequence or decreasing sequence of length $\sqrt{k}$. We can find this sequence in $\mathcal{O}(k \log k)$. Moreover, we can decompose the whole sequence into $\sqrt{k} \log k$ increasing and decreasing sequences. Therefore in our problem, we can create $\sqrt{n} \log n$ chains and connect them with $\sqrt{k} \log k$ additional segments. This solution uses $n + 2\sqrt{n} \log n$ segments and receives about 60 points.

$n + 6$ solution

Let us reconsider the spiral. We waste additional segments only if the bounding box contains less than 4 points (for example there's a point that is both the left-most and the top-most one). We can remove these points and put them in one of two chains (one going from the top-left corner to the bottom-right corner) and one going from the top-right corner to the bottom-left

one).

The algorithm is as follows:

- if a bounding box has 4 points in it, add them to the spiral, and remove these points,
- otherwise, find a point that lies on two sides of the bounding box, add it to the proper chain and remove it.

Then we have three objects (spiral and two chains) and we can use up to 2 segments to connect them to themselves and the origin. This solution gets about 95 points.

$n + 3$ solution

To come up with an even better solution, we need to add some small tricks, including considering all possible orders in which we can connect these objects and directions in which we can traverse them. These optimizations lead a solution with $n + 3$ segments, which gets 100 points.

Vision (Review)

Let's call a *right diagonal* a set of pixels that have coordinates $(r + a, c + a)$, where (r, c) is one of the pixels on the diagonal and a is an integer assuming values in some range. The difference between coordinates has the same value for all pixels lying on a diagonal: it's equal to $D = r - c$. For two right diagonals with such differences equal to D_1 and D_2, we say that the diagonals are at distance d if $|D_1 - D_2| = d$.

In a similar manner, let's call a *left diagonal* a set of pixels that have coordinates $(r + a, c - a)$. The sum of coordinates is equal to $D = r + c$ for all pixels on a left diagonal. Just as with right diagonals, two left diagonals are at distance d if $|D_1 - D_2| = d$.

Lemma. Distance between two pixels (r_1, c_1) and (r_2, c_2) is less than or equal to d if and only if both the distance between the left diagonals that contain the pixels is less than or equal to d and the distance between the right diagonals that contain the pixels is less than or equal to d.

Proof. The condition on diagonals can be written as follows:

$$|(r_1 - c_1) - (r_2 - c_2)| \le d,$$
$$|(r_1 + c_1) - (r_2 + c_2)| \le d.$$

This is equivalent to:

$$-d \le (r_1 - c_1) - (r_2 - c_2) \le d,$$
$$-d \le (r_1 + c_1) - (r_2 + c_2) \le d.$$

This in turn is equivalent to the following four double inequalities:

$$-d \le (r_1 - r_2) + (c_2 - c_1) \le d,$$
$$-d \le (r_2 - r_1) + (c_1 - c_2) \le d,$$
$$-d \le (r_1 - r_2) + (c_1 - c_2) \le d,$$
$$-d \le (r_2 - r_1) + (c_2 - c_1) \le d.$$

Finally, this is equivalent to $|r_1 - r_2| + |c_1 - c_2| \le d$, where the LHS is the definition of distance between pixels. The lemma has been proven.

The algorithm then is as follows:

1. Add an instruction for each diagonal (both left and right ones) indicating whether there is

a black pixel within the diagonal.

2. Add an instruction for each diagonal (both left and right ones) indicating whether there are two black pixels within the diagonal (combining OR and XOR might come in handy).

3. Using instructions from 1. and 2., for each block of $K+1$ consecutive left diagonals have an instruction that reports whether there are two black pixels within the block.

4. Do the same for right diagonals.

5. Add a gate that returns true if and only if there is at least one gate from 3. and at least one gate from 4. that both return true.

6. Repeat steps 3. — 5. for K instead of $K+1$.

7. The pixels are at distance K if and only if 5. reports true but 6. reports false.

The algorithm requires $\mathcal{O}(H+W)$ instructions which consume $\mathcal{O}(HW)$ inputs. Note, however, that there exist other completely different approaches with the same order of instructions and inputs used.

Sky Walking (Review)

Subtask 1

For each skywalk and each building, check if they intersect and if they do, add a new vertex for the intersection point. Additionally, put a node on the bottom of each of the buildings s, and g. Add edges between consecutive nodes on each building and also for consecutive nodes on each skywalk. Use Dijkstra to find the shortest path from the node on the bottom of building s to the node at the bottom of building g. Number of vertices and edges are $\mathcal{O}(NM)$, so the total complexity is $\mathcal{O}(NM \log NM)$.

Subtask 2

The solution for the previous subtask works here as well, however the graph must be built more efficiently. To do that, iterate in increasing order over heights which either contain a skywalk or the endpoint of a building. The goal is to keep a list of all buildings that are at least as tall as the current height. Given such list, for each skywalk, start from its left endpoint and move through the list. Each element is an intersection between that skywalk and a building. Hence you will at most visit 10 elements before reaching the right endpoint. Add a vertex for each intersection. The rest of the solution is the same as the last subtask. For maintaining the list of buildings, it is enough to start from a list of all buildings. Then at each height, after processing the skywalks for that height, remove the buildings that have an endpoint at that height. This can be done using a linked-list or a BST.

Subtask 3

When $s = 0$ and $g = n - 1$, it can be proven that skywalks are always traversed from left to right. Additionally when all of the buildings have the same height, it can be shown that there exists an optimal path where each skywalk is either not visited or is traversed completely until its right endpoint (though the entrance point to that skywalk might not be its left endpoint). Iterate over skywalks in increasing order of their right endpoint, breaking ties in favor of lower skywalks. The goal is to maintain the minimum cost to reach each skywalk's right endpoint. During the iteration, for each skywalk, update this value for the skywalks just above, and just below its right endpoint based on its own value. The answer is the minimum cost to reach the right endpoint of one of skywalks ending at $n - 1$ adding the cost of reaching to the bottom of

the building $n-1$.

Subtask 4

For this subtask, the solution is to use the same strategy as the first two subtasks and build a graph. Consider the graph in those two subtasks. Let e be a vertical edge connecting points (x, y_1) and (x, y_2) for some x, y_1, y_2 ($y_1 < y_2$). Edge e is called irrelevant if the point (x, y_2) lies strictly inside of some skywalk. Since $s = 0$ and $g = n - 1$, it can be proved that there exists a shortest path that does not pass through irrelevant edges. Hence, after removing the irrelevant edges from the graph, the length of the shortest path doesn't change. In the new graph, for each vertical edge, the top node is the endpoint of a skywalk. So, there are at most $\mathcal{O}(M)$ vertical edges left in the graph. This means at most $\mathcal{O}(M)$ vertices have at least one edge connected to them. Discarding the other vertices, the same approach as the first two subtasks can be followed on the new graph.

Subtask 5

The solution for this subtask is almost the same as the last subtask. However, for the previous theorem to hold, the skywalks must be adjusted. Specifically, for each skywalk between two buildings such as l, and r where $l < s < r$, divide it into 3 parts as follows:

- let a be the last building before s (including s) that is as tall as this skywalk.
- let b be the first building after s (including s) that is as tall as this skywalk.
- replace the skywalk with the following skywalks:
    - skywalk connecting buildings l and a.
    - skywalk connecting buildings a and b.
    - skywalk connecting buildings b and r.

The same adjustment must be done for all skywalks that intersect or pass over building g. Then the same solution as subtask 4 works.